Project Proposal

Project: Real-time IoT Data Pipeline (Cold Chain Fleet Telemetry)

Objective: To design and build an end-to-end real-time data streaming pipeline that monitors and processes IoT telemetry data from a logistics fleet. The goal is to track vehicle locations, monitor cargo conditions (temperature, weather impact), and provide real-time analytics to prevent supply chain disruptions.

Description: This project simulates a real-world scenario where a logistics company needs to monitor its fleet in real-time. Since physical IoT devices are not available, we will build a Python simulator that reads historical logistics data and streams it as live JSON payloads. The pipeline will ingest this data using a message broker, process it on the fly to detect delays or operational anomalies, store the time-series data efficiently, and visualize the fleet's status on a live interactive dashboard.

Tools & Design:

- **Data Source Simulation:** Python (Pandas) to read Excel datasets (Primary data and Refined sheets) and simulate live data streams.
- **Data Ingestion:** MQTT Broker to receive high-throughput, lightweight messages.
- **Data Processing:** Python Consumer scripts to subscribe to topics, clean, merge, and transform the data streams.
- **Storage:** A Time-Series Database optimized for real-time insert operations and time-based querying.
- **Visualization:** A dashboarding tool to display live tracking maps and key metrics.
- **Infrastructure:** Docker Compose to containerize and manage all services locally.

Technologies to Use:

- **Language:** Python
- **Message Broker:** Mosquitto MQTT / Apache Kafka (Optional, for scalability)
- **Database:** TimescaleDB (PostgreSQL extension for time-series)
- **Visualization:** Grafana
- **Deployment:** Docker & Docker Compose
- **Libraries:** pandas, paho-mqtt, psycopg2, sqlalchemy

Team Members & Responsibilities:

1- Youssef Elsayed Mohamed Ahmed (Team Leader)
   - *Responsibility:* Project management, defining the architecture, setting up the Docker Compose environment, and ensuring integration between all pipeline components.
2- Zakaria Ahmed Muhyiddin Almashd
   - *Responsibility:* Developing the Python IoT Simulator, handling data extraction from the source files, and implementing the MQTT Publisher.
3- Abdalaziz Ayman Ali Wahba
   - *Responsibility:* Developing the Python Stream Consumer, subscribing to MQTT topics, and implementing data cleaning and transformation logic.
4- Abdelmonem Khaled Abdelmonem Ahmed
   - *Responsibility:* Database Administration (DBA); designing the relational schema and configuring the TimescaleDB hypertables for optimal time-series storage.
5- David Fady Farg Tawfik
   - *Responsibility:* Writing the database ingestion scripts (connecting the Python Consumer to TimescaleDB) and handling error logging during data insertion.
6- Muhammad Ali Nassim Elnezami
   - *Responsibility:* Dashboard & Visualization; connecting Grafana to TimescaleDB, writing SQL queries for the dashboard, and designing the live map and metrics panels.

Week 1: *Project Setup and Data Simulation*

- **Youssef Elsayed:** Set up the GitHub repository, create the docker-compose.yml file with Mosquitto MQTT, and configure the local development environment.
- **Zakaria Ahmed:** Analyze the provided Excel dataset, identify the key columns for streaming, and write the initial Python script to read the data row by row.
- **Abdalaziz Ayman:** Research and test MQTT connection parameters in Python using the paho-mqtt library.

- **Abdelmonem Khaled:** Install and configure TimescaleDB locally, and start designing the Entity-Relationship Diagram (ERD).
- **David Fady:** Assist in data cleaning exploration to understand potential data quality issues before streaming.
- **Muhammad Ali:** Install Grafana via Docker and explore connection plugins for PostgreSQL/TimescaleDB.

Deliverables:

- A running Docker environment with Mosquitto MQTT.
- GitHub repository initialized with project structure.
- A working Python script that can read the dataset and print JSON payloads locally.

Week 2: Data Ingestion and Database Design

- **Zakaria Ahmed:** Finalize the Python IoT Simulator to publish JSON messages to the MQTT broker at regular intervals (e.g., 1 message per second).
- **Abdalaziz Ayman:** Create the Python Consumer script that successfully connects to the MQTT broker and receives the simulated messages.
- **Abdelmonem Khaled:** Execute SQL scripts to create the TimescaleDB database, tables, and hypertable structures required for the time-series data.
- **David Fady:** Write basic Python functions to connect to the PostgreSQL/TimescaleDB instance using psycopg2.
- **Youssef Elsayed:** Monitor the data flow from the Simulator to the Consumer and troubleshoot any network issues within the Docker containers.
- **Muhammad Ali:** Review the database schema with Abdelmonem to ensure the tables are optimized for the required Grafana queries.

Deliverables:

- Real-time data flowing successfully from the Python Simulator to the MQTT Broker.
- Database schema created and hypertables configured in TimescaleDB.

Week 3: *Data Processing and Storage Integration*

- **Abdalaziz Ayman:** Implement data transformation logic inside the Consumer script (adding processing timestamps, handling missing values).
- **David Fady:** Integrate the Consumer script with the database functions to enable continuous, real-time insertion of streamed data into TimescaleDB.
- **Abdelmonem Khaled:** Monitor database performance during continuous data ingestion and tune indexes if necessary.
- **Muhammad Ali:** Connect Grafana to the populated TimescaleDB instance and start building the initial panels (total trips, average speed).
- **Zakaria Ahmed:** Enhance the Simulator to introduce random anomalies (e.g., sudden temperature spikes or delays) for testing purposes.
- **Youssef Elsayed:** Conduct a mid-project review, ensure code is pushed to GitHub, and verify that the end-to-end flow is working without crashes.

Deliverables:

- A fully functional ingestion pipeline: Simulator -> MQTT -> Consumer -> TimescaleDB.
- Data being stored continuously in the database.

Week 4: *Visualization, Testing, and Documentation*

- **Muhammad Ali:** Finalize the Grafana dashboard, incorporating a live tracking map (using coordinates), time-series graphs, and alert indicators for delays or anomalies.
- **Abdelmonem Khaled & David Fady:** Write complex SQL aggregation queries (e.g., window functions) required by Grafana for advanced analytics.
- **Abdalaziz Ayman & Zakaria Ahmed:** Perform stress testing on the pipeline, document the Python code, and write instructions on how to run the scripts.
- **Youssef Elsayed:** Finalize the project documentation (README.md), record a demo video of the working pipeline, and prepare the final presentation.

Deliverables:

- A complete, interactive Grafana dashboard displaying real-time metrics.
- Final project source code pushed to GitHub with a comprehensive README.md.
- Final presentation and project handover.